

Projeto Integrador II

{ Desenvolvimento Estruturado }

Prof. Dr. Fernando T. Fernandes



```
highcharts-yaxis" data-z-index="2">...</g>  
highcharts-navigator-yaxis"  
'g>  
highcharts-crosshair highcharts-crosshair-thin undefined"  
visibility="hidden" d="M 778.5 45 L 778.5 494"></path>  
series-group" data-z-index="3">...</g>  
series-exporting-group" data-z-index="3">...</g>  
anchor="start" class="highcharts-title" data-z-index=  
anchor="middle" class="highcharts-subtitle" data-z-  
text>  
anchor="start" class="highcharts-caption" data-z-index=  
-axis-labels highcharts-xaxis-labels" data-z-index="7"  
-axis-labels highcharts-xaxis-labels highcharts-navig  
="7">...</g>  
-axis-labels highcharts-yaxis-labels" data-z-index="7"  
-axis-labels highcharts-yaxis-labels highcharts-naviga  
x="7"></g>  
range-selector-group" data-z-index="7" transform="tran  
legend" data-z-index="7" transform="translate(40,596)  
navigator" data-z-index="8" visibility="visible">...</g>  
highcharts-tooltip-0 highchar  
opacity="0"
```

Fintech

- A empresa precisa implementar a interface de consulta de saldo e depósito na conta dos clientes.
- Para isso, você deverá implementar uma estrutura que facilite a gestão de contas de clientes, facilitando a consulta ao saldo e a realização de depósitos.

Problemas

- ❑ Como representar as características de uma conta no código?
- ❑ Como garantir que todas as contas tenham a mesma estrutura ?
- ❑ Como realizar depósitos, saques e consulta de saldo?

Programação Orientada a Objetos (POO)

“A orientação a objetos permite descrever, por meio de trechos de código, objetos do mundo real e suas relações”

Forma de programação (paradigma) mais próxima a como pensamos e nos comunicamos!

Algumas linguagens com suporte a P00



E dezena de outras ...

Classe

Representação da estrutura, propriedade, comportamentos e ações de um objeto real ou abstrato.

Ex: Veículo, Animal, Produto, ContaCorrente, etc.

Classe da nossa fintech

```
class ContaCorrente {  
    numero  
    cliente  
    saldo  
}
```

Nome da classe

Atributos = Características

Problemas

- ✓ Como representar as características de uma conta no código?
- ❑ Como garantir que todas as contas tenham a mesma estrutura ?
- ❑ Como realizar depósitos, saques e consulta de saldo?

Como diferentes contas podem ter a mesma estrutura?

```
class ContaCorrente {
```

```
    numero
```

```
    cliente
```

```
    saldo
```

```
    constructor(numero, cliente, saldo = 0){
```

```
        this.numero = numero
```

```
        this.cliente = cliente
```

```
        this.saldo = saldo
```

```
    }
```

```
}
```



Construtores

Permitem inicializar
variáveis da classe

Problemas

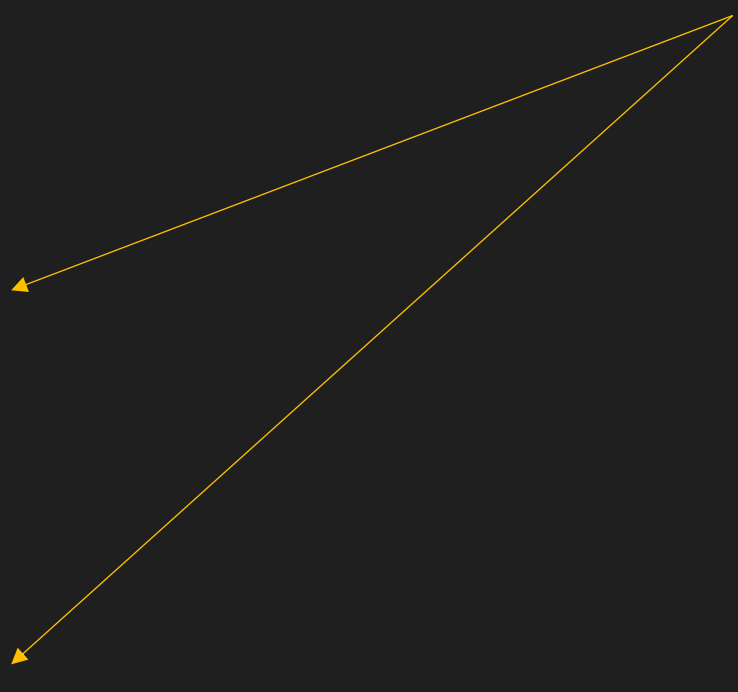
- ✓ Como representar as características de uma conta no código?
- ✓ Como garantir que todas as contas tenham a mesma estrutura ?
- ❑ Como realizar depósitos, saques e consulta de saldo?

Como verificar estoque e aplicar descontos?

```
class ContaCorrente {  
  
    numero  
    cliente  
    saldo  
  
    constructor(numero, cliente, saldo = 0){  
        this.numero = numero;  
        this.cliente = cliente;  
        this.saldo = saldo;  
    }  
  
    consultarSaldo(){  
        return `Saldo: R$ ${this.saldo.toFixed(2)}`;  
    }  
  
    depositar(valor){  
        this.saldo += valor;  
    }  
  
} // fim da classe
```

Métodos

Ações que podem ser executadas.



Problemas

- ✓ Como representar as características de uma conta no código?
- ✓ Como garantir que todas as contas tenham a mesma estrutura ?
- ✓ Como realizar depósitos, saques e consulta de saldo?

Classes vs Objetos



Classe

Projeto (molde)



```
class Produto {  
  constructor(nome,  
    preco, categoria) {  
    this.nome = nome;  
    this.preco = preco;  
    this.categoria = categoria;  
  }  
}
```



Objetos

Construção (instâncias)



Produtos reais com nomes, preços e categorias diferentes



Smartphone

R\$ 2500

Eletrônicos



Hambúrguer

R\$ 25

Alimentos



Tênis Esportivo

R\$ 180

Categorias
Roupas

Classes vs Objetos



Classe (ContaCorrente)

- Define os atributos
- Define os métodos
- É o molde

new

Objetos (instâncias)



conta1

numero: 1
cliente: "Ana"
saldo: 1000



conta2

numero: 2
cliente: "Carlos"
saldo: 2500

Classes vs Objetos

Classe (Projeto)

```
class ContaCorrente {  
  
    numero  
    cliente  
    saldo  
  
    constructor(numero, cliente, saldo = 0){  
        this.numero = numero  
        this.cliente = cliente  
        this.saldo = saldo  
    }  
  
    consultarSaldo() {  
        return this.saldo;  
    }  
}
```

Objeto (Construção)

```
const conta1 = new ContaCorrente(1, "Luiz" , 100.00)  
console.log(conta1.consultarSaldo());  
  
const conta2 = new ContaCorrente(2, "Ana" , 500.00)  
console.log(conta2.consultarSaldo());
```

Criação de objetos – conta-corrente.js

```
export class ContaCorrente {  
  
  numero  
  cliente  
  saldo  
  
  constructor(numero, cliente, saldo = 0){  
    this.numero = numero  
    this.cliente = cliente  
    this.saldo = saldo  
  }  
}
```

Classe ContaCorrente

Defina os atributos e o construtor

Criação de objetos – app.js

```
import { ContaCorrente } from "../conta-corrente.js"

const conta1 = new ContaCorrente(1234, "Luiz", 100);
const conta2 = new ContaCorrente(4321, "Ana", 1000);
const conta3 = new ContaCorrente(5555, "Roberta");

console.log("Conta do(a): " + conta1.nome + " criada com sucesso! ");
console.log("Conta do(a): " + conta2.nome + " criada com sucesso! ");
console.log("Conta do(a): " + conta2.nome + " criada com sucesso! ");
```

Criação de 3 objetos da classe

Executar ações dos objetos

Classe ContaCorrente (arquivo conta-corrente.js)

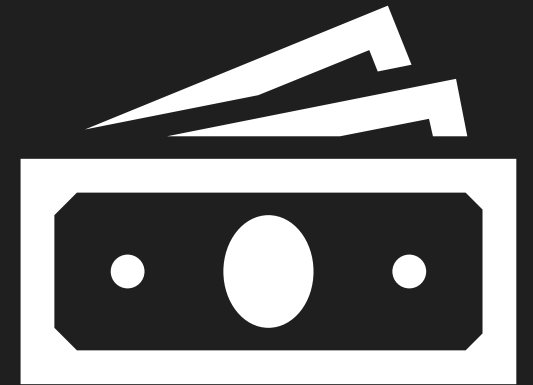
```
class ContaCorrente {  
  
  numero  
  cliente  
  saldo  
  
  constructor(numero, cliente, saldo = 0){  
    this.numero = numero;  
    this.cliente = cliente;  
    this.saldo = saldo;  
  }  
  
  consultarSaldo(){  
    return `Saldo: R$ ${this.saldo.toFixed(2)}`;  
  }  
  
  depositar(valor){  
    this.saldo += valor;  
  }  
  
}
```

Interagindo com objetos (arquivo app.js)

```
import { ContaCorrente } from "../conta-corrente.js"  
  
const conta1 = new ContaCorrente(1234, "Luiz", 100);  
const conta2 = new ContaCorrente(4321, "Ana", 1000);  
const conta3 = new ContaCorrente(5555, "Roberta");  
  
console.log("Cliente: " + conta3.cliente);  
  
//Depósito de 100 reais na conta da Roberta  
conta3.depositar(100.00);  
  
console.log("Saldo: " + conta3.consultarSaldo());
```

Exercício – Classes e Objetos Preparatório para AD0 1

1. Adicione um novo método chamado “sacar” que recebe um parâmetro ;
2. Crie um nova conta com o saldo inicial de 500 reais;
3. Realize um saque de 50 reais;
4. Exiba o saldo após o saque.



Sobrecarga de Métodos

- E se quiséssemos depositar um valor e incluir uma descrição?
- A sobrecarga de métodos permite que as ações do objeto sejam executadas de diferentes maneiras
- No Javascript, não há suporte nativo à sobrecarga, mas há alternativas com o uso do “rest parameter (...)”

Implementação de sobrecarga de métodos

Classe ContaCorrente (arquivo conta-corrente.js)

```
export class ContaCorrente{
  numero = 0;
  cliente = "";
  saldo = 0;

  constructor(numero, cliente, saldo=0){
    this.numero = numero;
    this.cliente = cliente;
    this.saldo = saldo;
  }

  sacar(...args) {
    if (args.length === 1 && typeof args[0] === "number") {
      // Um argumento
      this.saldo -= args[0];
    } else if (args.length === 2 && typeof args[0] === "number") {
      // dois argumentos
      this.saldo -= args[0];
      console.log("Saque referente a: " + args[1]);
    } else {
      throw new Error("Argumentos inválidos.");
    }
  }
}
```

Interagindo com objetos (arquivo app.js)

```
import { ContaCorrente } from "./conta-corrente.js"

const minhaConta = new ContaCorrente(1, "Fernando", 500.00);

//Acessando o método padrão
minhaConta.sacar(50);
console.log(minhaConta.saldo);

//Acessando o método sobrecarregado
minhaConta.sacar(25, "Hambúrguer");

console.log(minhaConta.saldo);
```



TypeScript

TypeScript

- Criado em 2012 pela Microsoft. Inclui:
 - **Tipagem Estática:** Adiciona tipos a variáveis o que auxilia na detecção de erros em tempo de compilação.
 - **Interfaces:** Funcionam como um contrato, garantindo o uso correto dos objetos.
 - **Herança:** Suporte a herança clássica com tipagem e modificadores de acesso.
 - **Recursos Javascript:** O TypeScript é um superset (superconjunto) do Javascript, portanto, é possível usar todos os recursos nativos do Javascript

TypeScript

- Adicionar o compilador de TypeScript

```
npm install -g typescript
```

```
C:\ProjetosJavascript\fintech.js>npm install -g typescript
```

```
added 1 package, and changed 1 package in 4s
```


Classes e Objetos em TypeScript

Classe ContaCorrente (arquivo conta.ts)

```
export class ContaCorrente {
  numero: number;
  cliente: string;
  saldo: number;

  constructor(numero: number, cliente: string, saldo: number = 0) {
    this.numero = numero;
    this.cliente = cliente;
    this.saldo = saldo;
  }

  consultarSaldo() {
    return `Saldo atual: R$ ${this.saldo.toFixed(2)}`;
  }

  depositar(valor: number) {
    this.saldo += valor;
  }

  sacar(valor: number, descricao?: string) {
    if (descricao === undefined) {
      // Um argumento
      this.saldo -= valor;
    } else {
      // Dois argumentos
      this.saldo -= valor;
      console.log("Saque referente a: " + descricao);
    }
  }
}
```

Classe App2 (arquivo app2.ts)

```
import {ContaCorrente} from "../conta";

const minhaConta = new ContaCorrente(1, "Fernando", 500.00);
console.log(minhaConta.consultarSaldo());

minhaConta.depositar(100.00);

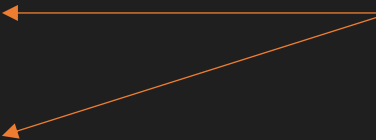
minhaConta.sacar(60.00);
minhaConta.sacar(250.00, "Luz");
minhaConta.sacar(60.00, "Celular");

console.log(minhaConta.consultarSaldo());
```

Executando o Código TS no Terminal

```
C:\ProjetosJavascript\fintech.js>tsc app2.ts  
C:\ProjetosJavascript\fintech.js>tsc conta.ts
```

Transpilação para Javascript



```
C:\ProjetosJavascript\fintech.js>node app2.js  
Saldo atual: R$ 500.00  
Saque referente a: Luz  
Saque referente a: Celular  
Saldo atual: R$ 230.00
```

Execução do Javascript transpilado



TypeScript - Playground

TS

TypeScript

Download

Docs

Handbook

Community

Playground

Tools

Em Pt

Playground

TS Config ▾

Examples ▾

Help ▾

v6.0.3 ▾

Run

Export ▾

Share

→

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

```
class ContaCorrente {
    numero: number;
    cliente: string;
    saldo: number;

    constructor(numero: number, cliente: string, saldo: number = 0) {
        this.numero = numero;
        this.cliente = cliente;
        this.saldo = saldo;
    }

    consultarSaldo() {
        return `Saldo atual: R$ ${this.saldo.toFixed(2)}`;
    }

    depositar(valor: number) {
        this.saldo += valor;
    }

    sacar(valor: number, descricao?: string) {
        if (descricao === undefined) {
            // Um argumento
            this.saldo -= valor;
        }
    }
}
```

JS

.D.TS

Errors

Logs

Plugins

```
"use strict";
class ContaCorrente {
    constructor(numero, cliente, saldo = 0) {
        this.numero = numero;
        this.cliente = cliente;
        this.saldo = saldo;
    }

    consultarSaldo() {
        return `Saldo atual: R$ ${this.saldo.toFixed(2)}`;
    }

    depositar(valor) {
        this.saldo += valor;
    }

    sacar(valor, descricao) {
        if (descricao === undefined) {
            // Um argumento
            this.saldo -= valor;
        }
        else {
            // Dois argumentos
            this.saldo -= valor;
            console.log("Saque referente a: " + descricao);
        }
    }
}
```



ADO 1 – Introdução a Orientação a Objetos

1. Crie um projeto **em TypeScript** no padrão “<SeuNomeSobreNome>AD01”. Ex: JoaoSilvaAD01
2. Escolha um animal que inicie com a mesma letra que seu nome (ex: Enzo – E):
Elefante, Esquilo, etc.
3. Defina **uma propriedade nome** e mais **pelo menos duas** propriedades de seu animal (O que o seu animal possui) **além do nome**
4. Crie um construtor que receba o nome do animal.
5. Crie pelo menos dois métodos referentes ao que o seu animal possa fazer.
6. Instancie dois objetos da sua classe e exiba o nome dos animais criados.
7. **Envie o código da Classe TypeScript, da transpilação em Javascript e da execução do código transpilado.**



Tipo de tarefa: **Individual** | **Formato:** Word | PDF



Tempo Estimado: **30 minutos**

Obrigado!



<https://www.youtube.com/@proffernandotfernandes>